

Uczelnia	Politechnika Bydgoska im. J. J. Śniadeckich
Wydział	Telekomunikacji, Informatyki i Elektrotechniki
Przedmiot	Sztuczna Inteligencja
Tytuł projektu	System AI do gry w Kuhn Poker oparty na sieci neuronowej i algorytmie genetycznym
Dokument	Sprawozdanie z postępów – Tydzień 1
Autorzy	[Imię i Nazwisko 1] [Imię i Nazwisko 2]
Grupa	4
Data	20 maja 2026
Prowadzący	dr inż. Tomasz Talaśka

Rok akademicki 2025/2026

1 Zakres prac w tygodniu 1

Celem pierwszego tygodnia było przygotowanie kompletnego szkieletu projektu: działającego silnika gry, mechanizmu uczenia (algorytm genetyczny), zestawu przeciwników odniesienia oraz jasno zdefiniowanego interfejsu pomiędzy częścią Osoby 1 (silnik i ewolucja) a częścią Osoby 2 (sieć neuronowa). Założeniem było, aby na koniec tygodnia system dało się uruchomić od początku do końca, nawet przed ukończeniem właściwej sieci neuronowej.

Podział obowiązków

- **Osoba 1:** silnik gry, kodowanie stanu, system ekspertowy, strategia Nasha, algorytm genetyczny, kontrakt agenta, testy jednostkowe, dokumentacja (sekcje 1–2).
- **Osoba 2:** implementacja sieci neuronowej zgodnej z kontraktem agenta (w trakcie), dokumentacja (sekcje 3–4).

2 Zrealizowane elementy (Osoba 1)

2.1 Silnik gry

Zaimplementowano pełne reguły Kuhn Poker w module `src/engine/kuhn.py`. Stan gry reprezentowany jest przez kartę gracza (liczba całkowita 1–3) oraz historię akcji (łańcuch znaków, np. "pb"). Zdefiniowano komplet historii terminalnych wraz z wypłatami oraz funkcję `play_hand`, która rozgrywa pojedyncze rozdanie między dwoma agentami.

2.2 Kodowanie stanu

Moduł `src/engine/state.py` zamienia stan gry na 12-wymiarowy wektor cech (one-hot karty oraz one-hot trzech pozycji historii). Dodano funkcje pomocnicze `build_state_dict` oraz `encode_from_dict` umożliwiające współpracę z enkoderem Osoby 2 w uzgodnionym formacie słownikowym.

2.3 Agenci odniesienia

W module `src/expert/baseline.py` zaimplementowano:

- `random_agent` – agent losowy,
- `always_pass_agent`, `always_bet_agent` – agenci skrajni,
- `expert_agent` – system ekspertowy oparty na regułach,
- `nash_agent_factory` – strategia równowagi Nasha (rodzina parametryzowana wartością $\alpha \in [0, 1/3]$).

2.4 Algorytm genetyczny

W module `src/ga/` zaimplementowano funkcję fitness (`fitness.py`) oraz pętlę ewolucji opartą na bibliotece DEAP (`evolution.py`). Wykorzystano selekcję turniejową, krzyżowanie typu *blend* oraz mutację gaussowską, z zachowaniem najlepszego osobnika (Hall of Fame).

2.5 Kontrakt agenta – punkt styku z Osobą 2

Kluczowym elementem tygodnia było ustalenie jednolitego interfejsu między częściami projektu. W module `src/agent/base.py` zdefiniowano kontrakt, który musi spełniać każda klasa agenta:

1. atrybut klasowy `WEIGHT_SIZE` – długość genomu;
2. konstruktor `__init__(self, weights)` – budowa agenta z wektora wag;
3. metoda `__call__(self, card, history) -> Action` – wywoływalność instancji.

Dzięki wywoływalności instancji agent neuronowy będzie zamiennikiem *drop-in* dla agentów funkcyjnych, a sama klasa może pełnić rolę fabryki w algorytmie genetycznym:

```
best, log = run_evolution(NeuralAgent.WEIGHT_SIZE, NeuralAgent, ...)
```

Jako referencyjną implementację kontraktu dostarczono klasę `LinearAgent` (polityka liniowa), która jednocześnie pełni rolę tymczasowego zamiennika sieci. Pozwala to uruchomić pełny cykl treningowy zanim Osoba 2 ukończy sieć neuronową.

2.6 Testy jednostkowe

Przygotowano 16 testów (`pytest`) pokrywających reguły gry, kodowanie stanu, agentów odniesienia oraz kontrakt agenta. Wszystkie testy przechodzą.

3 Wyniki wstępne

Uruchomiono pełny cykl treningu (50 generacji, populacja 60) z zastępczą polityką liniową. Wytrenowany agent oceniony został w bezpośrednich pojedynkach (po 2000 rozdań, na-przemienne pozycje):

Przeciwnik	Średnia wygrana / rozdanie
agent losowy	+0,30
agent zawsze-bet	+0,40
system ekspertowy	≈ 0 (wynik zbliżony)
strategia Nasha	$\approx -0,06$

Wynik przeciwko strategii Nasha (około $-0,06$ żetona na rozdanie) jest zbliżony z teoretyczną wartością gry $-1/18 \approx -0,0556$, co stanowi wstępne potwierdzenie poprawności silnika oraz funkcji fitness.

4 Plan na tydzień 2

- Osoba 2: implementacja sieci neuronowej (`NeuralAgent`) zgodnej z kontraktem; podmiana zastępczej polityki liniowej.
- Wspólnie: pierwsze pełne treningi neuroewolucji, strojenie hiperparametrów (rozmiar populacji, liczba generacji, σ mutacji).
- Osoba 1: rozbudowa systemu ekspertowego, przygotowanie skryptów do generowania wykresów zbieżności z pliku `logbook.json`.
- Dokumentacja: uzupełnienie sekcji 3 (implementacja sieci).

5 Stan repozytorium

```
kuhn-poker-ga/  
  src/  
    engine/    kuhn.py, state.py  
    expert/    baseline.py  
    agent/     base.py, linear_agent.py, nn_agent_template.py  
    ga/        fitness.py, evolution.py  
  tests/       test_engine.py, test_agent.py    (16 testow, wszystkie  
    zielone)  
  docs/        main.tex, sprawozdanie_tydzien1.tex  
  main.py, requirements.txt, README.md
```

Uruchomienie: `pip install -r requirements.txt`, następnie `python -m pytest tests/ -v` oraz `python main.py`.